



Bacharelado em Ciência da Computação
Componente: Computação Gráfica → Professor: Claudio Kleina
Aula 7 – Atividades focadas no desenvolvimento de jogos

Lista de Atividades – Preparatório para construção do projeto final da disciplina

Atividade 1 - O Efeito Mosaico (Texture Wrapping):

A Tarefa: Mudar as coordenadas UV do cubo de (1.0, 1.0) para (3.0, 3.0).

O Desafio: Veremos que a textura "estica". Devemos usar a documentação do OpenGL para descobrir os comandos GL_TEXTURE_WRAP_S e GL_TEXTURE_WRAP_T e configurá-los como GL_REPEAT.

Conceito ensinado: Como criar paredes de tijolos e gramados infinitos sem texturas gigantes.

Texto explicativo:

Na matemática geral da Computação Gráfica, chamamos as coordenadas da imagem 2D de **U** (Horizontal) e **V** (Vertical). Porém, os criadores do OpenGL decidiram usar as letras **S** e **T** para representar exatamente a mesma coisa.

S = Eixo Horizontal (equivalente ao U).

T = Eixo Vertical (equivalente ao V).

Portanto, GL_TEXTURE_WRAP_S significa *"Como a textura deve se comportar no eixo horizontal"*.

#####

Atividade 2 - O Problema: Passando dos Limites

Como visto na atividade anterior, o Mapeamento UV padrão vai de **0.0 a 1.0**. Mas e se você disser ao OpenGL para desenhar uma parede e mapear os vértices dela de **0.0 a 3.0**?

O OpenGL vai olhar para a sua imagem (que termina no 1.0) e não vai saber o que desenhar no espaço que sobra de 1.0 até 3.0. Ele precisa de uma regra! Esse comportamento é o que chamamos de **Wrap** (Embrulho/Envolvimento).

A Solução: GL_REPEAT (O Efeito Mosaico)

Quando você configura o Wrap para GL_REPEAT, você está dizendo ao OpenGL: *"Quando chegar no final da imagem (1.0), comece a desenhá-la de novo a partir do zero"*.

Se o seu mapa UV vai de 0.0 a 3.0 e você ativou o GL_REPEAT, a placa de vídeo vai desenhar a sua imagem **3 vezes** cabendo perfeitamente dentro daquele polígono.

É assim que criamos chãos gigantescos (como um campo de grama ou uma rua de paralelepípedos) em jogos sem precisar de uma imagem de textura que pese 50 Gigabytes. Nós pegamos uma imagem pequena de grama e dizemos para o OpenGL repeti-la 100 vezes (GL_REPEAT com UV indo até 100.0).

Como isso fica no código Python?

Para aplicar essa configuração, logo após carregar a sua textura e gerar o ID dela, você adiciona estas duas linhas:

```
# Configura o eixo horizontal (S/U) para repetir
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_REPEAT)
# Configura o eixo vertical (T/V) para repetir
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_REPEAT)
```

Outras opções divertidas para testar depois do GL_REPEAT:

GL_MIRRORED_REPEAT: Faz a mesma coisa que o repeat, mas a cada repetição a imagem fica de cabeça para baixo ou espelhada, criando padrões geométricos interessantes sem emendas óbvias.

GL_CLAMP_TO_EDGE: Em vez de repetir, ele pega a cor do último pixel da borda da imagem e estica essa mesma cor até o infinito.

#####

Atividade 3 - A Borda Esticada (Clamp to Edge):

A Tarefa: Usando o mesmo código do mosaico, alterar o wrap para GL_CLAMP_TO_EDGE. **Conceito ensinado:** Entender o que acontece quando o UV ultrapassa 1.0 e o motor gráfico precisa "inventar" pixels puxando a última linha da imagem (muito usado para resolver bordas pretas em texturas transparentes).

Texto:

O comando GL_CLAMP_TO_EDGE (em tradução livre: "Grampear na Borda" ou "Prender na Borda") é outra regra que você pode dar ao OpenGL para lidar com coordenadas UV que ultrapassam o limite normal de 0.0 a 1.0.

Para entender como ele funciona e por que é tão útil na indústria de jogos, vamos dividir a explicação em duas partes:

A. O Efeito Visual (A Borda Esticada)

Quando você configura as coordenadas de uma parede geométrica para irem de 0.0 até 3.0, o OpenGL precisa preencher o espaço que sobra entre 1.0 e 3.0.

Se você usar GL_REPEAT (Mosaico), ele desenha a imagem inteira novamente.

Se você usar **GL_CLAMP_TO_EDGE**, ele se recusa a repetir a imagem. Em vez disso, ele pega **exatamente a última fileira de pixels** da borda da imagem e a "estica" infinitamente até cobrir o resto do polígono 3D.

O resultado visual é que a sua textura aparece perfeitamente uma vez e, do lado dela, você vê listras contínuas de cor que foram arrastadas da borda.

B. O Conceito Ensinado: Por que usar isso? (O problema da Borda Preta)

Você raramente vai querer esticar a borda de uma imagem no meio da tela de propósito. A grande utilidade do GL_CLAMP_TO_EDGE está na **correção de artefatos visuais** (bugs gráficos) quando lidamos com texturas com fundo transparente.

Imagine o seguinte cenário:

Você tem a imagem de uma árvore em um fundo transparente para colocar no seu jogo.

Você está usando o filtro GL_LINEAR para que o jogo fique com as bordas suaves.

O Problema do GL_REPEAT (Padrão): Quando o OpenGL tenta suavizar o pixel que está na **extrema borda superior** da imagem (UV 1.0), ele precisa fazer uma média com o pixel vizinho de cima. Mas não há nada acima dele! Se a textura estiver configurada para repetir (GL_REPEAT), o OpenGL "dá a volta" e usa os pixels da **borda inferior** da imagem para fazer essa média matemática. O resultado? O topo da sua árvore transparente ganha uma linha preta ou colorida misteriosa flutuando acima dela (conhecido como *Texture Bleeding* ou Sangramento de Textura).

A Solução com GL_CLAMP_TO_EDGE: Ao usar o Clamp, você diz ao OpenGL: "**Não dê a volta na imagem!**". Quando ele precisar suavizar a borda superior, ele vai simplesmente duplicar o pixel transparente da própria borda superior infinitamente. Isso garante que as texturas 2D recortadas fiquem com as bordas perfeitas e totalmente invisíveis, sem "puxar" lixo visual do outro lado da imagem.

Como aplicar no código:

Exatamente no mesmo lugar onde você colocou o GL_REPEAT no exercício anterior, você altera para o Clamp:

```
# Configura o eixo horizontal (S/U) para prender na borda
```

```
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_CLAMP_TO_EDGE)
```

Configura o eixo vertical (T/V) para prender na borda

```
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_CLAMP_TO_EDGE)
```

#####

Atividade 4 - O Dado de 6 Faces (Texture Atlas):

A Tarefa: Pegar uma única imagem que contém o desenho das 6 faces de um dado de RPG desdobradas. Modificar o array `uv_coords` para que cada face do cubo mostre apenas um dos números.

Conceito ensinado: Otimização de memória. Ensinar que em jogos 3D reais não carregamos 10 texturas para uma casa, mas sim uma única "Cartela de Texturas" (Texture Atlas).

Atividade 5 - Alternância de Filtros em Tempo Real:

A Tarefa: Programar o clique do **Botão Direito** do mouse para alternar a configuração de textura em tempo real de `GL_NEAREST` (pixelado) para `GL_LINEAR` (suave). **Conceito ensinado:**

Recarregamento dinâmico de parâmetros na GPU e compreensão visual dos filtros de minificação/magnificação.

#####

Atividade 6 - Zoom Interativo com o Scroll:

A Tarefa: Implementar a função de callback do scroll (`glfw.set_scroll_callback`). Usar o movimento da rodinha do mouse para alterar o eixo Z da função `glTranslatef` (aproximando ou afastando os objetos). **Conceito ensinado:** Controle de campo de profundidade. Devemos garantir que a variável de zoom tenha limites lógicos (para não atravessar o cubo ou inverter a câmera).

A. Como o Scroll Funciona no GLFW

A roda do mouse (scroll) não funciona como um botão comum (pressionado/solto). Ela gera um evento de "deslocamento". A função `glfw.set_scroll_callback` escuta esse evento e nos entrega duas variáveis: `xoffset` (rolagem horizontal, rara) e **yoffset** (rolagem vertical).

- Se você roda a bolinha para **frente**, o `yoffset` vale **1**.
- Se você roda a bolinha para **trás**, o `yoffset` vale **-1**.

B. A Ilusão do Zoom (Eixo Z)

No OpenGL clássico, a câmera está sempre cravada na origem do universo (0, 0, 0). Para enxergar um cubo que também nasce no (0, 0, 0), nós não movemos a câmera para trás; **nós empurramos o universo inteiro para o fundo da tela** usando o eixo Z negativo: `glTranslatef(0.0, 0.0, -5.0)`.

Ao atrelar o valor do scroll (`yoffset`) a essa variável Z, nós criamos a ilusão de Zoom:

- Scroll pra frente `rightarrow` Soma +1 no Z `rightarrow` O Z vai para -4.0 (O cubo se aproxima da câmera).
- Scroll pra trás `rightarrow` Soma -1 no Z `rightarrow` O Z vai para -6.0 (O cubo se afasta da câmera).

C. O Perigo: Por que precisamos de "Limites Lógicos"?

Se você não colocar um limite matemático na sua variável de Zoom, o usuário vai continuar rolando a bolinha do mouse para frente.

O Z vai passar de -2.0 para -1.0, para 0.0 e depois para 1.0. Quando isso acontece, ocorrem dois "bugs" visuais clássicos:

1. **Near Clipping (Corte da Geometria):** A câmera possui uma lente teórica (o `gluPerspective`). Se o objeto chegar perto demais (ex: cruzando a marca de -0.1), o OpenGL corta a face da frente do cubo, permitindo que você veja oco do objeto por dentro.
2. **Atravessar o Objeto:** Quando o Z se torna positivo (1.0), significa que você empurrou o cubo para *trás* da sua própria nuca. O cubo desaparece da tela.

Como fica no Código

Resolvemos isso usando a lógica de limite mínimo e máximo (o que chamamos em programação de função *Clamp*).

Variável global para guardar a distância do universo

`zoom_z = -5.0`

`def callback_scroll(window, xoffset, yoffset):`

`global zoom_z`

`velocidade_zoom = 0.5`

`# Altera o zoom baseado no movimento da rodinha`

`zoom_z += yoffset * velocidade_zoom`

`# =====`

`# OS LIMITES LÓGICOS (Clamp)`

`# =====`

`# Não deixa chegar perto demais (evita atravessar o cubo)`

`if zoom_z > -2.0:`

`zoom_z = -2.0`

`# Não deixa afastar para sempre (evita que o cubo vire um pixel)`

`elif zoom_z < -15.0:`

`zoom_z = -15.0`

`# E dentro do loop principal do while:`

`# glTranslatef(0.0, 0.0, zoom_z)`

`#####`

Atividade 7 - Câmera Pan (Arrastar a tela):

A Tarefa: Modificar a lógica do mouse: o Botão Esquerdo continua girando o cubo, mas segurar o **Botão do Meio (Scroll Click)** e arrastar o mouse deve mover a cena inteira para a esquerda/direita/cima/baixo (translação em X e Y). **Conceito ensinado:** Desacoplamento de rotação e translação na matriz de visualização.

Esse é um dos exercícios mais recompensadores, porque é o momento em que a aplicação deixa de parecer um "trabalho de faculdade" e passa a ter os controles idênticos aos de um software profissional de modelagem 3D (como Blender, Maya ou Unity).

A palavra Pan (abreviação de *Panning*) vem do cinema e significa fazer um movimento panorâmico — ou seja, deslizar a câmera horizontalmente ou verticalmente em seu próprio eixo, sem alterar o ângulo de visão ou a distância do objeto.

O Conceito: Desacoplamento da Matriz de Visualização

Quando os começamos a programar em 3D, tendemos a jogar todos os comandos de transformação juntos no código sem pensar na ordem. O problema é que em OpenGL, a ordem das matrizes importa (não é comutativa).

Se você aplicar a Translação e a Rotação na ordem errada, elas ficam "acopladas" (uma afeta o comportamento da outra).

- A Ordem Certa (Desacoplada): Movemos o universo nos eixos globais X e Y (Pan) e, independentemente de onde o objeto esteja na tela, a rotação continua girando o objeto em torno do seu próprio centro geométrico.
- A Ordem Errada (Acoplada): O objeto é movido para o lado e, quando você tenta girá-lo, ele começa a "orbitar" a origem do mundo como se estivesse amarrado a um barbante gigante, saindo da tela.

Na prática do código OpenGL, para que o Pan (arrastar a tela) e o Orbit (girar o cubo) funcionem em perfeita harmonia, lemos de baixo para cima: primeiro aplicamos a rotação, depois a translação.

Como implementar na prática (O Código)

Para realizar essa tarefa, o precisamos evoluir a lógica de escuta do mouse. Não podemos mais ter apenas um `mouse_pressionado = True`. Precisamos saber qual botão está pressionado.

Veja a lógica de resolução:

1. Variáveis separadas para Rotação e para Pan (Translação)

```
rotacao_x, rotacao_y = 0.0, 0.0
```

```
pan_x, pan_y = 0.0, 0.0
```

2. Rastreadores de estado para cada botão

```
mouse_esq_pressionado = False
```

```
mouse_meio_pressionado = False
```

```
ultimo_x, ultimo_y = 0.0, 0.0
```

```
def callback_botao_mouse(window, button, action, mods):
```

```
    global mouse_esq_pressionado, mouse_meio_pressionado
```

```
    # Atualiza o estado do botão esquerdo (Para girar)
```

```
    if button == glfw.MOUSE_BUTTON_LEFT:
```

```
        mouse_esq_pressionado = (action == glfw.PRESS)
```

```
    # Atualiza o estado do botão do meio/scroll (Para o Pan)
```

```
    elif button == glfw.MOUSE_BUTTON_MIDDLE:
```

```
        mouse_meio_pressionado = (action == glfw.PRESS)
```

```
def callback_movimento_mouse(window, xpos, ypos):
```

```
    global rotacao_x, rotacao_y, pan_x, pan_y, ultimo_x, ultimo_y
```

```
    dx = xpos - ultimo_x
```

```
    dy = ypos - ultimo_y
```

```
    # Se o botão ESQUERDO estiver segurado, alteramos o ÂNGULO
```

```
    if mouse_esq_pressionado:
```

```
        rotacao_y += dx * 0.5
```

```
        rotacao_x += dy * 0.5
```

```

# Se o botão do MEIO estiver segurado, alteramos a POSIÇÃO X e Y
# (Dividimos por um valor maior, ex: 100, para o movimento não ser ultra rápido)
elif mouse_meio_preionado:
    pan_x += dx * 0.01
    pan_y -= dy * 0.01 # Invertemos o Y porque o eixo Y da tela desce, mas o do 3D sobe

ultimo_x = xpos
ultimo_y = ypos

# =====
# DENTRO DO LOOP DE RENDERIZAÇÃO (O SEGREDO DA MATRIZ)
# =====
# 1. Aplicamos o Pan e o Zoom juntos na mesma matriz de translação principal
# pan_x e pan_y movimentam a tela, o -5.0 afasta a câmera para podermos enxergar
glTranslatef(pan_x, pan_y, -5.0)

# 2. Depois aplicamos a Rotação Global
glRotatef(rotacao_x, 1, 0, 0)
glRotatef(rotacao_y, 0, 1, 0)

# 3. Desenha os objetos aqui...

```

#####

Atividade 8 - A Lanterna do Mouse (Spotlight Móvel):

A Tarefa: Substituir a rotação da cena pelo controle da luz. Onde o mouse estiver apontando na tela (convertido de 2D para 3D), a luz da cena (GL_LIGHT0) deve seguir.

Conceito: Mapeamento de tela para coordenadas de mundo (Screen-to-World space). Ao passar o mouse pela textura da Terra, veremos o relevo sendo iluminado dinamicamente. Substituir a rotação da cena pelo controle da luz. Onde o mouse estiver apontando na tela (convertido de 2D para 3D), a luz da cena (GL_LIGHT0) deve seguir.

Conceito ensinado: Mapeamento de tela para coordenadas de mundo (Screen-to-World space). Ao passar o mouse pela textura da Terra, veremos o relevo sendo iluminado dinamicamente.

$$X_{norm} = \left(\frac{X_{mouse}}{Largura_{tela}} \right) \times 2 - 1$$

$$Y_{norm} = - \left(\left(\frac{Y_{mouse}}{Altura_{tela}} \right) \times 2 - 1 \right)$$

(Nota: Invertemos o Y multiplicando por um sinal negativo porque na tela os pixels crescem para baixo, mas no 3D o eixo Y cresce para cima).

Se a sua câmera de perspectiva (ou ortogonal) está enxergando uma área que vai do X = -5.0 até X = 5.0, basta multiplicar o valor normalizado por 5.

A Lâmpada do OpenGL: GL_LIGHT0

O OpenGL possui 8 luzes padrão (GL_LIGHT0 a GL_LIGHT7). Quando posicionamos uma luz, o comando exige um vetor de 4 números [x, y, z, w].

O segredo ensinado aqui é a letra **W** (o quarto valor):

- Se **W = 0.0**, a luz é tratada como o Sol (Direcional). Ela vem do infinito e X, Y, Z representam apenas o ângulo dela.
- Se **W = 1.0**, a luz é tratada como uma Lâmpada (Posicional). Ela tem uma posição X, Y, Z física dentro da cena. É isso que queremos para a nossa "lanterna" do mouse.

O Código

Veja como integrar a matemática dentro do callback do mouse e conectar à luz:

```
# Variáveis globais da luz no mundo 3D
```

```
luz_x = 0.0
```

```
luz_y = 0.0
```

```
def callback_movimento_mouse(window, xpos, ypos):  
    global luz_x, luz_y
```

```
    # 1. Converte Pixels para Normalizado (-1 a 1)
```

```
    x_norm = (xpos / 800.0) * 2.0 - 1.0
```

```
    y_norm = -((ypos / 600.0) * 2.0 - 1.0) # Y invertido!
```

```
    # 2. Escala para o tamanho do nosso mundo 3D
```

```
    # (Supondo que a câmera está vendo de -4.0 a 4.0 unidades)
```

```
    luz_x = x_norm * 4.0
```

```
    luz_y = y_norm * 3.0
```

```
# =====
```

```
# DENTRO DO LOOP DE RENDERIZAÇÃO
```

```
# =====
```

```
# Habilita a iluminação
```

```
glEnable(GL_LIGHTING)
```

```
glEnable(GL_LIGHT0)
```

```
# Posiciona a luz (X e Y vêm do mouse, Z é fixado na frente dos objetos, W=1.0 faz dela  
uma lanterna posicional)
```

```
posicao_da_luz = [luz_x, luz_y, 3.0, 1.0]
```

```
glLightfv(GL_LIGHT0, GL_POSITION, posicao_da_luz)
```

```
# Desenha a Esfera Terrestre aqui...
```

O Resultado Visual

Quando rodar isso com a textura do mapa-múndi e a esfera girando, iremos ver algo incrível.

Por usarmos o `gluQuadricNormals(quadric, GLU_SMOOTH)` no desenho da esfera, o OpenGL sabe exatamente a curvatura da Terra. Conforme movemos o mouse sobre o continente sul-americano, a África e a Europa mergulharão na escuridão (sombra), enquanto o Brasil ficará brilhantemente iluminado pela luz pontual. É a introdução perfeita à computação de **Normais**, a base para mapas de relevo (*Bump/Normal Maps*) em motores gráficos modernos.

```
#####
```

Atividade 9 - Inspeção Detalhada (Double Click Zoom):

A Tarefa: Implementar um cronômetro na lógica do clique. Se dermos "duplo clique" sobre o objeto, a câmera deve animar-se (transição suave ao longo do tempo) aproximando a câmera até focar na textura do bloco. **Conceito ensinado:** Interpolação linear (LERP) matemática acionada por eventos.

Este exercício marca a transição de um programa "duro" e mecânico para uma aplicação com **fluidez e polimento**. Animações suaves são o que separam um protótipo de um software profissional (seja um jogo ou a interface do seu celular).

Para resolver essa tarefa, precisamos conectar três conceitos distintos: Medição de Tempo, Mudança de Estado e Matemática Interpolada.

Aqui está a explicação detalhada de como essa engrenagem funciona:

1. O Cronômetro: Detectando o Duplo Clique

O sistema operacional não diz ao OpenGL: *"Ei, o usuário deu um duplo clique!"*. Precisamos construir essa lógica do zero. Para isso, usamos a função `glfw.get_time()`, que nos diz há quantos segundos a aplicação está rodando.

A Lógica:

1. Quando o usuário clica a primeira vez, nós guardamos aquele exato segundo em uma variável (ex: `tempo_ultimo_clique = 12.5`).
2. Quando o usuário clica novamente, verificamos o tempo atual (ex: 12.7).
3. Subtraímos um do outro: Se a diferença for menor que 0.3 segundos, bingo! Temos um duplo clique.

O Erro Comum: Animação Bloqueante (For Loop)

Quando você pede fazer a câmera ir da posição $Z = -5.0$ (Longe) para $Z = -2.0$ (Perto) suavemente, o primeiro instinto é fazer isso:

O JEITO ERRADO DE ANIMAR (Trava o programa)

```
for i in range(-5, -2):  
    camera_z = i  
    time.sleep(0.1)
```

Se fizermos isso, a tela inteira vai congelar. Nada vai ser desenhado, o mouse vai parar de responder, e depois de alguns segundos o cubo vai "teletransportar" para a tela. Em Computação Gráfica, **toda animação deve acontecer dentro do ciclo de vida natural do Frame** (o loop while principal). A câmera deve dar apenas "um passinho" por quadro renderizado.

A Solução: Interpolação Linear (LERP)

LERP é uma função matemática que encontra um valor intermediário entre dois pontos em uma reta, baseado em uma porcentagem de tempo.

A fórmula matemática oficial do LERP é:

$$\text{Valor} = A + (B - A) \times t$$

Onde A é a origem, B é o destino, e t é o progresso (de 0.0 a 1.0).

No entanto, no desenvolvimento de jogos, nós usamos um **"Truque de Lerp Assintótico"** (ou Suavização Exponencial). Ele não precisa de um cronômetro de animação e cria um efeito lindo onde a câmera começa rápida e vai freando suavemente quando chega perto do alvo.

O truque é simples: A cada *frame*, a câmera anda uma porcentagem fixa da distância que *ainda falta* para o alvo.

Como fica no Código

Precisaremos de duas variáveis de câmera: onde ela está (`camera_z`) e para onde ela deve ir (`alvo_z`).

```
import glfw
```

```
# Variáveis globais
```

```
tempo_ultimo_clique = 0.0
```



```

camera_z = -5.0 # Onde a câmera está agora
alvo_z = -5.0 # Para onde ela quer ir

def callback_botao_mouse(window, button, action, mods):
    global tempo_ultimo_clique, alvo_z

    if button == glfw.MOUSE_BUTTON_LEFT and action == glfw.PRESS:
        tempo_atual = glfw.get_time()

        # Verifica se o tempo entre este clique e o anterior é menor que 0.3s
        if tempo_atual - tempo_ultimo_clique < 0.3:
            print("Duplo Clique Detectado!")

            # Muda o ALVO da câmera (Muda de Estado)
            # Se estava longe, manda para perto. Se estava perto, manda para longe.
            if alvo_z == -5.0:
                alvo_z = -2.0 # Foca no bloco
            else:
                alvo_z = -5.0 # Volta para a visão geral

        # Atualiza a memória do tempo para o próximo clique
        tempo_ultimo_clique = tempo_atual

# =====
# DENTRO DO LOOP PRINCIPAL DO JOGO (while)
# =====

# A MÁGICA DO LERP ACONTECE AQUI, TODO FRAME!
# A fórmula: Atual = Atual + (Alvo - Atual) * Velocidade
velocidade_suavizacao = 0.05
camera_z += (alvo_z - camera_z) * velocidade_suavizacao

# Aplica a câmera matemática no universo 3D
glTranslatef(0.0, 0.0, camera_z)

# Desenha os objetos...

```

O mais legal de ensinar essa técnica é que percebemos que o `alvo_z` e o `camera_z` estão completamente desacoplados.

O evento do mouse **não move a câmera**, ele apenas **muda a intenção** (o alvo). Quem faz o trabalho pesado de deslizar a câmera perfeitamente a cada milissegundo é a fórmula matemática no fundo do loop. Esse conceito de "Programação Reativa a Estados" é a base da arquitetura da interface do iOS, do Android e da Unreal Engine.

#####

Atividade 10 - Água Deslizante (Animação UV):

A Tarefa: Desenhar um plano grande (chão) e aplicar uma textura de água. Antes de chamar o `glVertex`, somar uma pequena fração (baseada no tempo: `glfw.get_time()`) à coordenada 'U' ou 'V'. **Conceito ensinado:** Texturas dinâmicas. Como jogos como Mario 64 fazem lava e rios parecerem fluir infinitamente.

Este exercício introduz uma das técnicas mais antigas, brilhantes e eficientes da história dos videogames: o **UV Scrolling** (Deslizamento de UV).

É com essa técnica que jogos clássicos como *Super Mario 64* (na fase *Lethal Lava Land*), *The Legend of Zelda: Ocarina of Time*, e até jogos modernos, criam a ilusão de rios, cachoeiras, esteiras rolantes e lava fluindo infinitamente.

Aqui está a explicação detalhada do "truque" e de como devemos implementá-lo:

O Problema: Como fazer um rio infinito?

Se você tem um plano 3D que representa o chão de um rio e quer que a água se mova para a direita, a primeira ideia lógica seria usar `glTranslatef` para mover o chão 3D inteiro para o lado. Mas isso cria dois problemas:

1. O rio sairia do cenário e deixaria um buraco negro para trás.
2. Você precisaria de um modelo 3D gigantesco para durar a fase toda.

A Solução Inteligente: Deslizar o "Papel de Presente"

Em vez de mover a geometria (o objeto 3D real), nós deixamos o chão totalmente parado.

O que nós movemos é **a forma como a textura está embrulhada nele**.

Lembre-se que as coordenadas UV dizem qual parte da imagem vai em qual vértice do polígono. Se, a cada quadro do jogo (frame), nós dissermos ao OpenGL para "puxar" a textura um pouquinho para o lado, a água parecerá correr.

A Matemática com o Tempo (`glfw.get_time()`)

A função `glfw.get_time()` retorna um número que cresce infinitamente desde que o programa abriu (Ex: 1.0, 1.1, 1.2, 1.3...).

Se somarmos esse tempo diretamente à coordenada horizontal **U**, a leitura da imagem vai se deslocando.

- **Frame 1 (Tempo 0.0):** Começa lendo o U do 0.0.
- **Frame 60 (Tempo 1.0):** Começa lendo o U do 1.0.
- **Frame 120 (Tempo 2.0):** Começa lendo o U do 2.0.

O Pulo do Gato (GL_REPEAT): Lembra do primeiro exercício sobre Mosaico? Como a textura da água está configurada com `GL_REPEAT`, quando o UV ultrapassa 1.0 (ou 2.0, 100.0, etc.), o OpenGL não estica a imagem nem dá erro; ele simplesmente desenha a imagem da água novamente desde o início. Isso cria um loop visual contínuo e perfeito!

Como fica no Código

Precisaremos calcular um valor de "deslocamento" no início do loop e somá-lo **apenas** nas coordenadas do eixo que ele quer que a água flua (U para horizontal, V para vertical).

```
# =====
```

```
# DENTRO DO LOOP PRINCIPAL DO JOGO (while)
```

```
# =====
```

```
# 1. Pega o tempo atual e multiplica por uma velocidade
```

```
# (Se não multiplicar por 0.5, a água vai correr rápido como um jato!)
```

```
deslocamento_u = glfw.get_time() * 0.5
```

```
glEnable(GL_TEXTURE_2D)
```

```
glBindTexture(GL_TEXTURE_2D, textura_agua)
```

```
# 2. Desenha o chão da água (Um quadrado grande)
```

```
glBegin(GL_QUADS)
```

```
# Canto Inferior Esquerdo
```

```
glTexCoord2f(0.0 + deslocamento_u, 0.0)
```

```
glVertex3f(-3.0, -1.0, 3.0)
```

```
# Canto Inferior Direito (A textura base vai até 3.0 para repetir a imagem 3 vezes)
glTexCoord2f(3.0 + deslocamento_u, 0.0)
glVertex3f( 3.0, -1.0, 3.0)

# Canto Superior Direito
glTexCoord2f(3.0 + deslocamento_u, 3.0)
glVertex3f( 3.0, -1.0, -3.0)

# Canto Superior Esquerdo
glTexCoord2f(0.0 + deslocamento_u, 3.0)
glVertex3f(-3.0, -1.0, -3.0)

glEnd()
```

O impacto visual desse exercício é gigantesco. Ao rodar o código, vemos uma correnteza realista e contínua.

A maior lição que tiramos daqui como futuros desenvolvedores é a de **Otimização Gráfica**: ele acabou de criar uma animação complexa e infinita de água sem alterar um único vértice do modelo 3D e gastando zero poder de processamento (a matemática da textura ocorre instantaneamente na GPU). É a essência do desenvolvimento de jogos de alta performance!

#####

Atividade 11 - Terra e Nuvens (Multi-texturing Analógico):

A Tarefa: Carregar duas texturas: o mapa-múndi e um mapa de nuvens em PNG (com transparência). Desenhar a esfera terrestre. Ativar o `glEnable(GL_BLEND)`. Desenhar uma segunda esfera, 1% maior que a primeira, aplicando a textura das nuvens. **O Desafio:** Fazer a esfera da terra girar em uma velocidade e a das nuvens girar um pouco mais devagar.

Este exercício introduz a criação de **Camadas Visuais (Layers)** e ao gerenciamento da **Pilha de Matrizes**, técnicas fundamentais para criar gráficos mais realistas, como uma atmosfera planetária, escudos de energia ou vidros sujos.

Aqui está a explicação detalhada da física e da matemática por trás dessa técnica:

A Ilusão da Atmosfera (A Esfera 1% Maior)

Em motores gráficos simples, não tentamos misturar a imagem da Terra e das nuvens em uma única textura via código (isso seria muito pesado e limitante). O "truque" da indústria é desenhar duas geometrias separadas.

A Primeira Esfera (A Terra): Desenhada normalmente com o raio 1.0.

A Segunda Esfera (As Nuvens): Desenhada por cima, com o raio 1.01 (1% maior).

Por que exatamente 1% maior? Se você desenhar a esfera das nuvens com o exato mesmo tamanho (1.0) da esfera da Terra, o OpenGL sofrerá de um erro gráfico horrível chamado **Z-Fighting** (Briga de Z). A placa de vídeo não consegue decidir quem está na frente de quem, fazendo a tela piscar loucamente. O 1.01 garante que as nuvens fiquem sempre como uma "casca" externa.

A Mágica da Transparência (GL_BLEND)

Imagens no formato .PNG possuem um quarto canal de cor chamado **Alpha** (RGBA), que dita o quão transparente cada pixel é. Porém, por padrão, o OpenGL ignora o Alpha e desenha fundos transparentes como preto ou branco.

Para que a parte vazia das nuvens deixe a Terra aparecer por baixo, precisamos:

Ligar o motor de mistura: `glEnable(GL_BLEND)`.

Ensinar a fórmula de transparência padrão ao OpenGL usando o comando `glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA)`. Isso diz: *"Pegue a cor da nuvem, multiplique pela opacidade dela, e some com a cor da Terra que já está desenhada no fundo"*.

O Desafio: Rotações Independentes (`glPushMatrix`)

Se tentarmos colocar um `glRotatef` para a Terra e logo em seguida outro `glRotatef` para as nuvens, a matemática de matrizes vai somar os dois. As nuvens girariam na velocidade da Terra *mais* a velocidade das nuvens, como se estivessem coladas.

Para fazer objetos orbitarem de forma independente, ensinamos o conceito de **Pilha de Matrizes (Push e Pop)**:

`glPushMatrix()`: Salva a posição/rotação atual do universo.

Fazemos a rotação da Terra e a desenhamos.

`glPopMatrix()`: Descarta a rotação da Terra e restaura o universo para a estaca zero.

Repetimos o processo para as nuvens, com uma velocidade diferente.

A Estrutura do Código

Veja como o raciocínio ficaria traduzido em código dentro do loop principal:

```
tempo_atual = glfw.get_time()
```

```
# Calcula velocidades separadas
```

```
rotacao_terra = tempo_atual * 20.0 # Rápida
```

```
rotacao_nuvens = tempo_atual * 5.0 # Devagar
```

```
# =====
```

```
# 1. DESENHA A TERRA (Sólida)
```

```
# =====
```

```
glPushMatrix()          # Salva o universo parado
```

```
glRotatef(rotacao_terra, 0, 1, 0) # Gira só a Terra
```

```
desenhar_esfera(textura_terra, raio=1.0)
```

```
glPopMatrix()           # Volta o universo para o estado parado
```

```
# =====
```

```
# 2. DESENHA A ATMOSFERA (Transparente)
```

```
# =====
```

```
glEnable(GL_BLEND)
```

```
glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA)
```

```
glPushMatrix()          # Salva o universo parado de novo
```

```
glRotatef(rotacao_nuvens, 0, 1, 0) # Gira só as Nuvens
```

```
desenhar_esfera(textura_nuvens, raio=1.01) # Esfera 1% maior!
```

```
glPopMatrix()
```

```
glDisable(GL_BLEND) # Desliga para não afetar o resto do jogo
```

```
#####
```

Tarefa 12 - O Quarto de Vidro (Skybox):

A Tarefa: Desenhar um cubo gigante (onde a câmera fica dentro dele). Desligar a escrita de profundidade (`glDepthMask`). Aplicar texturas de paisagem nas faces internas do cubo. O mouse não gira os objetos, gira a "cabeça" da câmera. **Conceito ensinado:** Criação de

cenários de fundo infinitos (Skyboxes), técnica fundamental em qualquer motor gráfico moderno (Unity, Unreal).

Este é, sem dúvida, o "truque de mágica" mais icônico da Computação Gráfica. É exatamente assim que jogos como *GTA*, *Minecraft* ou *Skyrim* criam a ilusão de um mundo infinito com um céu distante, nuvens e montanhas no horizonte, sem fritar a placa de vídeo do computador.

Vamos destrinchar a anatomia de um **Skybox** (Caixa de Céu) e entender a genialidade matemática por trás dele.

O Cubo Gigante e as Faces Internas

Normalmente, desenhamos um cubo e a câmera fica do lado de fora olhando para ele. No Skybox, nós colocamos a câmera exatamente na coordenada (0,0,0) e desenhamos um cubo ao redor dela.

Para que a textura não fique invisível, precisamos inverter a ordem de desenho dos vértices ou desligar o *Culling* (o sistema que esconde a parte de trás das faces), garantindo que a textura da paisagem seja "colada" no lado de dentro das paredes do cubo.

A Mágica: `glDepthMask(GL_FALSE)` (O Z-Buffer)

Essa é a parte mais importante de toda a lição. O *Depth Buffer* (ou Z-Buffer) é a memória que o OpenGL usa para saber quem está na frente de quem. Se você desenha uma montanha a 50 metros e uma árvore a 10 metros, ele usa o Depth Buffer para desenhar a árvore por cima da montanha.

O Problema sem a máscara: Se o seu Skybox for um cubo de tamanho 100, e você tentar desenhar um avião que voou até a distância 120, o avião vai atravessar a parede do céu e sumir! O universo pareceria uma caixa fechada.

A Solução (`glDepthMask`): Ao executar `glDepthMask(GL_FALSE)`, você desliga a *escrita* de profundidade. Você diz ao OpenGL: *"Desenhe este cubo do céu, mas minta para a placa de vídeo. Não registre a profundidade dele no Z-Buffer"*.

Como resultado, o OpenGL desenha a paisagem no fundo da tela e "esquece" que ela está lá. Quando você ligar a profundidade de novo e for desenhar o resto do jogo (árvores, prédios, o avião no horizonte), **tudo** será desenhado por cima do céu, não importa o quão longe esteja. Isso cria a ilusão perfeita de distância infinita.

A Câmera FPS (First Person Shooter)

Até agora, estávamos girando a matriz do universo ao redor da origem. Em um Skybox, nós não orbitamos o cubo; nós giramos a "cabeça" da câmera no próprio eixo (Pitch para cima/baixo, Yaw para os lados). Para a ilusão ser perfeita, o Skybox deve estar sempre ancorado na mesma posição da câmera, não importa para onde o jogador ande.

A Estrutura do Código no OpenGL

Veja como essa lógica é montada dentro do loop de renderização do jogo:

```
# =====
```

```
# 1. DESENHO DO SKYBOX (Cenário de Fundo)
```

```
# =====
```

```
# Desliga a escrita de profundidade para o céu parecer infinito
```

```
glDepthMask(GL_FALSE)
```

```
glPushMatrix()
```

```
# Aplica apenas a rotação do mouse (cabeça do jogador olhando ao redor)
```

```
glRotatef(rotacao_x_mouse, 1, 0, 0) # Olha para cima/baixo
```

```
glRotatef(rotacao_y_mouse, 0, 1, 0) # Olha para a esquerda/direita
```

```
# Desenha o cubo texturizado com a paisagem nas faces internas
```

```
desenhar_cubo_skybox(textura_paisagem)
```

```
glPopMatrix()
```

```
# Liga a profundidade de volta! Fundamental para o resto do jogo não bugar.  
glDepthMask(GL_TRUE)
```

```
# =====
```

```
# 2. DESENHO DO JOGO REAL
```

```
# =====
```

```
glPushMatrix()
```

```
# Aqui aplica a câmera normal de jogo, translações, o personagem, a Terra, etc.
```

```
glTranslatef(pos_x, pos_y, pos_z)
```

```
desenhar_personagens_e_mundo()
```

```
glPopMatrix()
```

Com a máscara ativada (Comportamento correto): O motor gráfico ignora o tamanho real do Skybox. Ele pinta o fundo e o objeto 3D sempre aparece por cima, perfeito.

Com a máscara desativada (O Bug): O motor gráfico percebe que o objeto está mais longe que a parede do Skybox, e a parede esconde o objeto. A ilusão se quebra.

Bom estudo!